Correzione del C02, l'istruzione if e digitalRead

Lezione 10 — 1MAK, SST-INF-01c — Le due palline su otto LED, operatori di confronto, if, leggere un pin con digitalRead, indentazione

06/11/2026

1 Da dove ripartiamo

La lezione è divisa in due parti. Prima si **corregge** il compito per casa **C02** (le due palline su otto LED), poi si fa un **passo avanti**: Arduino impara a **prendere decisioni** con l'istruzione `if` e a **leggere** un pin con `digitalRead`.

Attenzione

Leggete bene il testo. Prima di scrivere il codice, leggete **tutto il testo** della richiesta, anche la parte su come si consegna. Non fate di testa vostra sperando che, per caso, coincida con quello che era richiesto.

2 Correzione del C02: le due palline

2.1 Che cosa chiedeva il testo

"Immagina che ogni luce accesa sia una **pallina**. Le due palline partono dai **due lati**, vanno verso il **centro**, **sbattono** e tornano indietro, ripetendo il movimento **all'infinito**." Quindi sono accese **due luci per volta**. Nel testo c'era anche il disegno dei primi quattro passi: è disegnata **solo la prima metà**, poi il movimento torna indietro.

2.2 Il circuito

Il circuito doveva essere **già pronto** da casa: in aula non serve rifarlo, serve seguire il ragionamento. Si costruisce così, in pochi minuti:

1. si prende un **Arduino Uno** e una **breadboard**;
2. si collegano **subito GND e 5 V** ai rail della breadboard (nero al meno, rosso al più), così si è a posto e non ci si pensa più;
3. si mette un **resistore da 220 Ω** e un **LED**; poi si **selezionano** i due componenti insieme, si **copiano** e si incollano fino ad averne otto;
4. si aggiungono subito i fili che dal **catodo** vanno al **GND**: senza di loro il circuito **non può funzionare**;
5. infine si collegano i pin, partendo dall'alto: **dal pin 2 al pin 9**, un **colore diverso** per ogni filo.

Cambiare il colore dei LED non è necessario: fate come volete, ma non è lì che va speso il tempo.

Nota

Il testo del compito usava i pin **dal 3 al 10**; in aula si è partiti dal **2**, fino al 9. Si può partire dal pin che si vuole: basta **cambiare i numeri** nel codice in modo coerente.

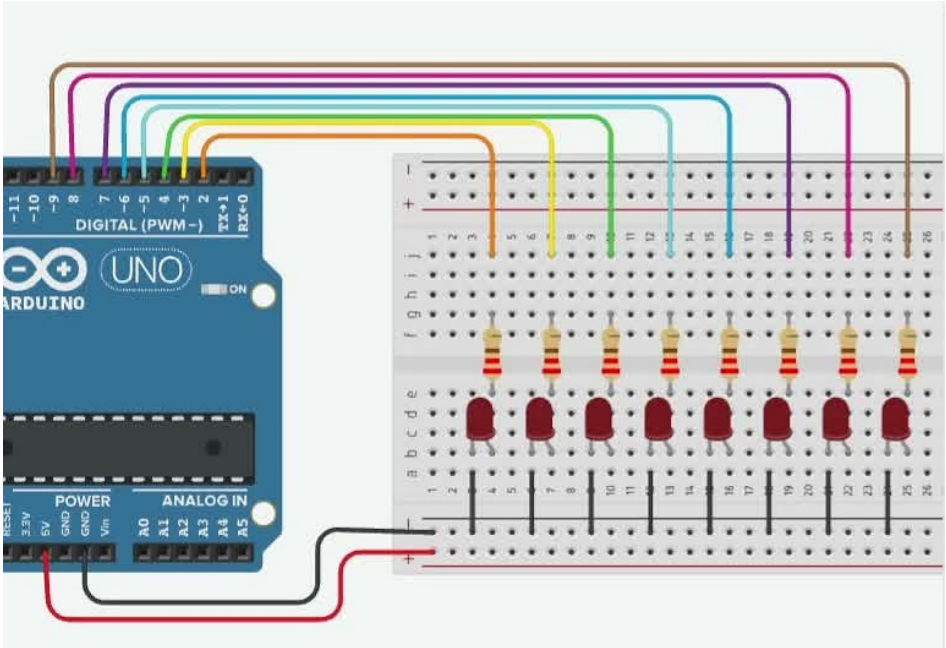


Figura 1: Il circuito del C02 in Tinkercad: otto fili colorati dai pin 2-9, otto resistori da 220 Ω, otto LED, i catodi verso il rail del GND.

2.3 Il codice, passo dopo passo

setup. Si **pulisce** il codice di esempio che Tinkercad propone. Poi si impostano **tutti i pin usati** come `OUTPUT`, perché devono **pilotare dei LED**: dal 2 al 9. Il modo più veloce: si scrive `pinMode(2, OUTPUT);`, si fa **copia e incolla** sette volte e si cambiano i numeri (3, 4, 5, 6, 7, 8, 9).

Passo 1: il primo e l'ultimo. Si accendono insieme il pin **2** e il pin **9**, con due `digitalWrite(..., HIGH)`. Ogni passo dura **200 millisecondi**: prima di fare qualunque altra cosa si aspetta con `delay(200)`.

Nota

Simulate a ogni modifica. Ogni volta che aggiungete un pezzo, avviate la simulazione e controllate che **fino a quel punto** il codice faccia quello che deve. Gli errori si trovano subito, quando il pezzo nuovo è piccolo.

Passo 2: il secondo e il penultimo. Prima si **spengono** i due LED accesi: gli stessi due `digitalWrite`, ma con `LOW` invece di `HIGH`. Subito dopo, **senza attesa**, si accendono il **3** e l'**8**. Si aspettano altri 200 ms e li si spegne.

Nota

Rallentare per controllare. A 200 ms la sequenza è troppo veloce per controllarla a occhio. Mentre si scrive il programma conviene mettere tutti i `delay` a **500 ms**: si vede bene che si accendono prima i LED esterni e poi quelli interni. Alla fine si riportano tutti a **200 ms**, come chiede il testo.

Passo 3: il terzo e il terzultimo. Si va avanti di uno da tutte e due le parti: dal 3 si passa al **4**, dall'8 si passa al **7**. Si accendono il 4 e il 7, si aspetta, poi si spengono **proprio il 4 e il 7**.

Attenzione

Spegnete gli stessi LED che avete acceso. Se si accende il 7 ma poi, per una svista, si spegne il 5, in simulazione il LED **7 resta sempre acceso**. Quando un LED non si spegne mai, cercate nel codice il suo `digitalWrite(..., LOW)`: quasi sempre il numero del pin è sbagliato.

Passo 4: le palline al centro. Si accendono il **5** e il **6**, si aspetta, si spengono.

Attenzione

Il punto e virgola. Ogni istruzione (`pinMode`, `digitalWrite`, `delay`) finisce con il **punto e virgola**. Se ne manca anche uno solo, per esempio dopo un `delay`, il programma **non parte**.

Il ritorno. Le palline sono arrivate al centro e adesso devono **tornare indietro**. Non serve scrivere niente di nuovo: basta **ripetere il passo 3** (il 4 e il 7) e poi **il passo 2** (il 3 e l'8). Si possono riscrivere, ma è più veloce **copiarli e incollarli** dopo il passo 4. Il passo 1 non va ripetuto in fondo: ci pensa `loop`, che **riparte dall'inizio** e riaccende il 2 e il 9.

Per rendere il codice più leggibile si possono **separare i passi** con una riga vuota.

```
void setup()
{
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
}

void loop()
{
  // passo 1: il primo e l'ultimo
  digitalWrite(2, HIGH);
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(2, LOW);
  digitalWrite(9, LOW);

  // passo 2: il secondo e il penultimo
  digitalWrite(3, HIGH);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(8, LOW);

  // passo 3: il terzo e il terzultimo
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(7, LOW);

  // passo 4: le palline al centro
  digitalWrite(5, HIGH);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, LOW);

  // ritorno: di nuovo il passo 3
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(7, LOW);

  // ritorno: di nuovo il passo 2
  digitalWrite(3, HIGH);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(8, LOW);
}
```

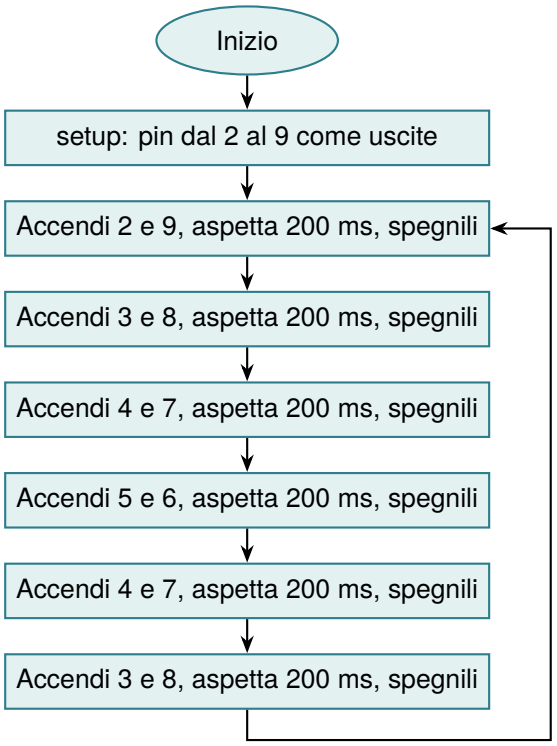


Figura 2: Le due palline: quattro passi verso il centro, due per tornare indietro, poi `loop` riparte dal primo e dall'ultimo LED.

2.4 Gli errori visti ai banchi

- **Il LED "esplode" in simulazione.** Se Tinkercad mostra un LED bruciato, controllate il **valore dei resistori**: devono essere tutti da **220 Ω**. Un resistore lasciato a **1 Ω** non limita la corrente e il LED si brucia. Contate anche i componenti: 8 LED vogliono **8 resistori**, né uno di più né uno di meno.
- **La sequenza non torna indietro.** Se le palline arrivano al centro e poi ripartono dai lati, manca il **ritorno** (passi 3 e 2 ripetuti). Rileggete il testo: il movimento è **avanti e indietro**.

Nota

Il codice e lo schema sono qui: provate a rifarli **a casa** in Tinkercad, per essere sicuri di riuscirci da soli. Non è una consegna: serve a voi, per capire e vedere che tutto funziona.

Attenzione

Si lavora anche a casa, e con un computer. La correzione in aula non è il momento per ricopiare tutto il circuito: il lavoro va fatto prima, a casa. In aula non c'è il tempo di rifare tutto. Il **telefono** non è uno strumento supportato per Tinkercad: per i compiti serve un **computer**.

3 Le schede che prendono decisioni

Fino ad adesso abbiamo collegato dei LED ad Arduino e li abbiamo pilotati: con `pinMode` decidiamo che un pin è in **uscita**, così possiamo collegarci un LED e decidere **noi**, nel programma, se

accenderlo o spegnerlo. Ma la scheda **non interagisce con noi**: la accendiamo e lei accende i LED nella sequenza scritta, sempre uguale, un po' come le **luci di Natale**.

Le schede elettroniche però possono anche **interagire** con noi e **cambiare il loro modo di funzionare** in base a quello che succede fuori. Pensate al **microonde**, al **forno**, alla **lavatrice**: non funzionano sempre allo stesso modo. Ci sono dei **pulsanti**, e in base a quello che premiamo la scheda decide se avviare un lavaggio veloce o un lavaggio intenso, se scaldare o non scaldare.

La scheda quindi **prende una decisione**: funzionare in un modo o nell'altro. E la prende in base a un **confronto** tra due elementi: da lì decide se fare una cosa o farne un'altra.

4 Gli operatori di confronto, di nuovo

Gli **operatori di confronto** li abbiamo già visti nella dispensa A04. Attenzione: il **modulo** `%` **non** è un operatore di confronto, perché fa un **calcolo** e dà un numero. Gli operatori di confronto danno **solo vero o falso**, e sono **sei**:

Operatore	Significato
`>`	maggiore
`<`	minore
`>=`	maggiore o uguale
`<=`	minore o uguale
`==`	uguaglianza
`!=`	diverso (disuguaglianza)

Esempio

Creo una variabile `int a` con il valore **7** e una variabile `int b` con il valore **5**.

- `a > b`: 7 è maggiore di 5? **Vero**.
- `a < b`: 7 è minore di 5? **Falso**.

5 L'istruzione if

Il linguaggio ha un **costrutto**, cioè una struttura, fatto apposta per **prendere decisioni** in base a un confronto. Si chiama `if` (in inglese, "se").

Definizione

if — istruzione **condizionale**. Si scrive `if`, poi tra **parentesi tonde** la **condizione**; sotto, a capo, si apre una **graffa**, più in giù la si chiude. Dentro le graffe va il codice da eseguire.

- Se la condizione è **vera**, il codice dentro le graffe **viene eseguito**.
- Se la condizione è **falsa**, il codice dentro le graffe **viene saltato**.

```
if (condizione)
{
  digitalWrite(...);
  delay(...);
}
```

Il programma, dentro `loop`, esegue le sue istruzioni una dopo l'altra: istruzione 1 (per esempio un `digitalWrite`), istruzione 2 (un `delay`), istruzione 3, istruzione 4... A un certo punto bisogna **decidere** se eseguire un pezzo di codice oppure no, in base a come vogliamo che si comporti la macchina. Lì si mette l'`if` con la sua condizione: `a > b`, `a == b` o qualunque altro confronto.

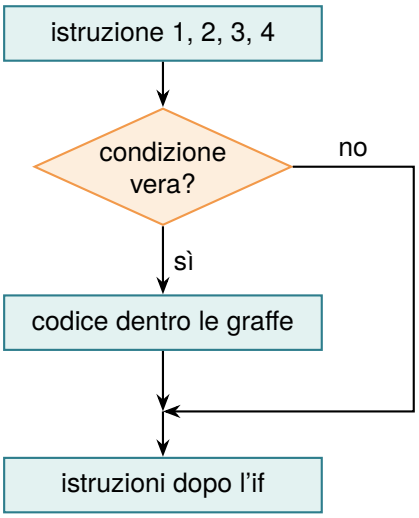


Figura 3: Che cosa fa l'if: se la condizione è vera esegue il blocco tra le graffe, altrimenti lo salta.

Attenzione

Niente punto e virgola dopo l'if. L'`if` **non** vuole il punto e virgola, né dopo la parentesi tonda della condizione né dopo la graffa. Vale anche per gli altri **costrutti** che vedremo (per esempio i cicli): sui **comandi** (`pinMode`, `digitalWrite`, `delay`...) il punto e virgola **ci va**, sui **costrutti non ci va**. Molti dimenticano il punto e virgola dove serve; qui è il contrario: se lo mettete dopo la condizione, l'if **non funziona** come vi aspettate.

Nota

Perché il punto e virgola rovina l'if. `if (a == HIGH);` vuol dire "se `a` è alto, **non fare niente**": il punto e virgola chiude subito l'if. Il blocco tra le graffe che viene dopo non dipende più dalla condizione e viene eseguito **sempre**. Il programma si compila lo stesso, quindi l'errore non viene segnalato: si vede solo dal comportamento sbagliato.

6 Le dispense vanno studiate: verifica di martedì 17/11

Gli **operatori di confronto** erano nella dispensa A04: le dispense sono la memoria di quello che abbiamo fatto, e **vanno studiate**, non solo conservate.

Attenzione

Verifica di firmware: martedì 17 novembre 2026 (se venerdì 13 la classe non sarà pronta, verrà spostata più avanti). Comprende **tutte le dispense** di firmware, dalla A01 fino a quelle della prossima settimana: tipi di dato, operatori, Arduino, `setup` e `loop`, `pinMode`, `digitalWrite`, `delay`, operatori di confronto, `if`. La prossima settimana si finisce l'argomento e venerdì si fanno esercizi.

7 Leggere un pin: digitalRead

Con un `if` la scheda sceglie se eseguire o no un pezzo di codice. Ma se le variabili le decidiamo noi all'inizio del programma (`int a = 7;`), il confronto dà **sempre lo stesso risultato**, e la scelta è sempre la stessa. Serve un modo per far dipendere la variabile da qualcosa che succede **fuori** dalla scheda.

Esiste un comando che, invece di **scrivere** su un pin (e accendere un LED), lo **legge**. Se per scrivere si usa `digitalWrite`, per leggere si usa `digitalRead` (con la **R maiuscola**). Tra parentesi si mette il **numero del pin** da leggere.

```
int a;
a = digitalRead(10);
```

Quando scriviamo su un LED, `HIGH` vuol dire acceso e `LOW` spento. Anche in lettura un filo può essere **alto** (`HIGH`, collegato ai 5 V) o **basso** (`LOW`, collegato a 0 V, il GND). `digitalRead` restituisce proprio questo. Così la macchina può **cambiare il suo funzionamento** e scegliere se eseguire o no un pezzo di codice in base a una **modifica hardware** fatta sulla scheda.

Definizione

INPUT — un pin che deve essere **letto** si imposta in `setup` come **ingresso**: `pinMode(10, INPUT);`. I pin dei LED restano `OUTPUT`.

8 La prova in Tinkercad: un filo al posto del pulsante

8.1 Il circuito

1. Un **Arduino** e una breadboard, con **GND e 5 V** collegati ai rail come sempre.
2. Un **LED** con il suo resistore da **220 Ω**, collegato al **pin 3**: serve per **controllare** che cosa succede.
3. Un **filo** dal **pin 10**: è il pin che leggeremo. L'altro capo del filo va su uno dei rail: ai **5 V** oppure al **GND**.

8.2 Il codice

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(10, INPUT);
}

void loop()
{
  int a = digitalRead(10);

  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }

  if (a == LOW)
  {
    digitalWrite(3, LOW);
  }
}
```

Non abbiamo scritto tanta roba: abbiamo usato quello che abbiamo appena visto.

- In `setup`: il **3** è un'uscita (c'è il LED), il **10** è un ingresso.
- In `loop`: dichiariamo la variabile `a` e le diamo il valore letto dal **pin 10**.
- Primo `if`: se `a` è **alto**, accendiamo il LED. Secondo `if`: se `a` è **basso**, lo spegniamo.

Qui non abbiamo usato una variabile `b`: confrontiamo `a` **direttamente** con `HIGH`. Avremmo potuto creare una variabile `b` con il valore `HIGH`, ma sarebbe uno **spreco di memoria** per niente.

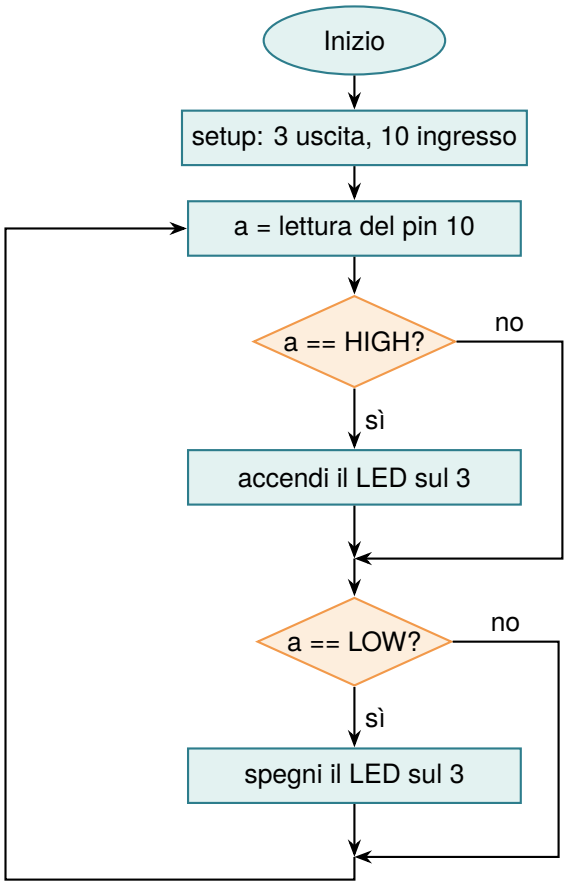


Figura 4: Il programma con `digitalRead`: a ogni giro di `loop` legge il pin 10 e decide se accendere o spegnere il LED.

8.3 Spostare il filo

- Il filo del pin 10 va sui **5 V** (il rail rosso): il pin è **alto**, a vale `HIGH`, il primo `if` è vero e il LED **si accende**.
- Si **ferma la simulazione** (in Tinkercad, mentre la simulazione è in corso, i fili non si possono spostare) e si porta il filo sugli **0 V** (GND): il pin è **basso**, a vale `LOW`, il secondo `if` è vero e il LED **resta spento**.

Il **codice non è cambiato**: è cambiato solo dove è collegato il filo. Spostando il filo da 5 V a 0 V decidiamo se il LED è acceso o spento.

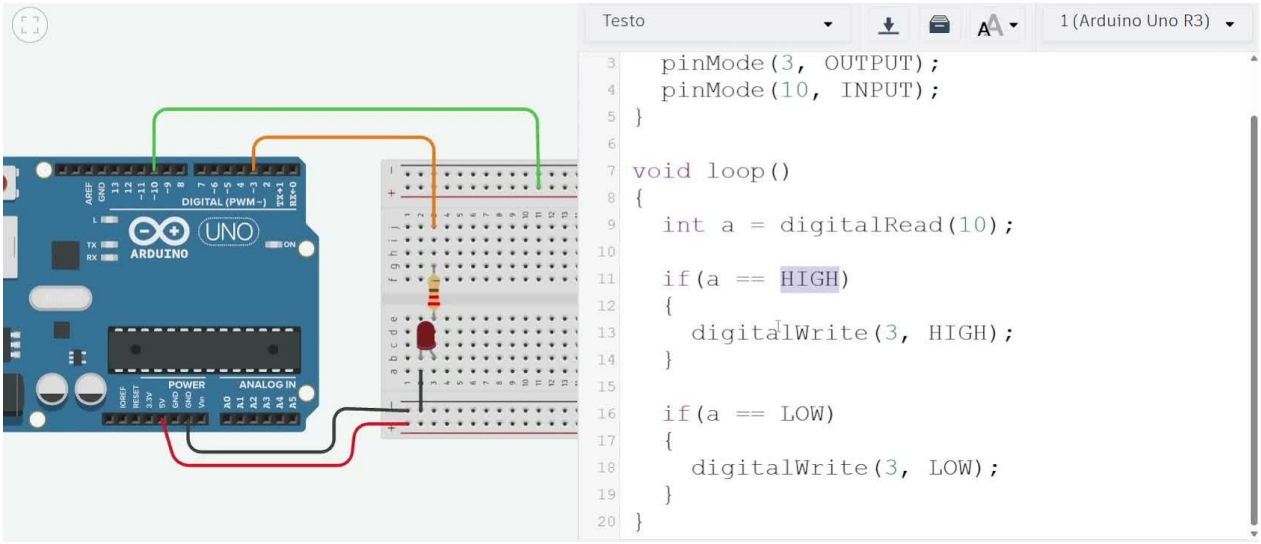


Figura 5: In Tinkercad: il LED sul pin 3, il filo verde dal pin 10 verso il rail, e il codice con i due if (prima versione: alto = acceso).

8.4 Il funzionamento lo decidiamo noi

Si può fare anche il **contrario**: accendere il LED quando a è **basso** e spegnerlo quando è **alto**. Basta scambiare `HIGH` e `LOW` nelle due condizioni.

```
if (a == LOW)
{
  digitalWrite(3, HIGH);
}

if (a == HIGH)
{
  digitalWrite(3, LOW);
}
```

Adesso, con il filo sugli 0 V il LED **si accende**, con il filo sui 5 V **si spegne**. Il funzionamento **lo decide chi scrive il programma**.

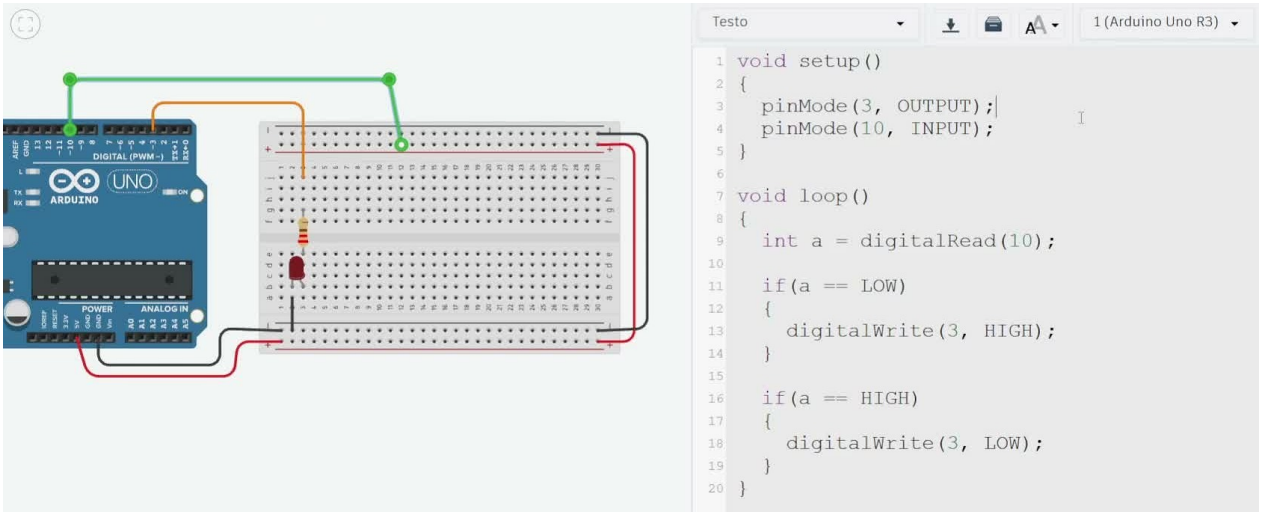


Figura 6: La versione invertita: il LED si accende quando il pin 10 è basso.

Accendere un LED spostando un filo non serve a molto: per accendere una lampadina non serve un Arduino. Ma è il modo più semplice per capire **come funziona l'if**. Al posto del filo, più avanti, ci sarà un **pulsante**, come quelli del microonde o della lavatrice.

Nota

Un ingresso non va lasciato scollegato. Il filo del pin 10 deve essere sempre collegato ai **5 V** o al **GND**. Un pin di ingresso **scollegato** (si dice "flottante") non è né alto né basso: `digitalRead` può leggere `HIGH` o `LOW` **a caso**. Per questo, quando useremo i pulsanti, servirà anche un resistore che tenga il pin a un valore preciso quando il pulsante non è premuto.

9 Il codice ordinato: l'indentazione

Guardate il codice qui sopra: tutto quello che sta **dentro un paio di graffe** è spostato un po' **a destra**. Il contenuto di `setup` e di `loop` è spostato di un passo; il contenuto di ogni `if`, che sta dentro `loop`, è spostato di **un passo in più**.

Definizione

Indentazione — ogni volta che si apre una graffa, cioè si crea un **contenitore**, tutto il testo al suo interno si sposta a destra. Non si usano gli spazi: si usa il **tasto di tabulazione** (**Tab**), quello **a sinistra della Q**.

Quanti spazi fa il tasto Tab dipende dalla **configurazione dell'editor**: in **Tinkercad** il rientro è piccolo, **2 spazi**; in altri editor sono 3, 4, a volte 5. Di solito si può configurare.

A che cosa serve? A capire **a colpo d'occhio** che un pezzo di codice è un **blocco unico**: si vede subito dove inizia e dove finisce ogni contenitore. Chi scrive tutto **attaccato a sinistra** perde questa visione: succede spesso che si **perda una graffa** per strada, che non veda l'errore o non capisca dove sta sbagliando, perché il codice **non è organizzato**.

Senza indentazione

```
void loop()
{
int a = digitalRead(10);
if (a == HIGH)
{
digitalWrite(3, HIGH);
}
}
```

Con indentazione

```
void loop()
{
  int a = digitalRead(10);
  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }
}
```

Nota

Alcuni editor (per esempio l'Arduino IDE, con **Ctrl+T**) sistemano l'indentazione **da soli**. L'editor di Tinkercad no: lì va fatta **a mano**, con il tasto Tab, mentre si scrive.

10 Cosa ricordare di questa lezione

- **Leggere tutto il testo** di un esercizio prima di iniziare.
- C02: per passare da un passo al successivo si **spengono** i due LED accesi e si accendono i due nuovi; il **ritorno** ripete i passi 3 e 2; `loop` riparte dal passo 1.
- Simulare **a ogni modifica**; rallentare i `delay` per controllare, poi rimetterli al valore richiesto.
- Gli operatori di confronto sono **sei** e danno **vero o falso**; il modulo `%` non è un operatore di confronto.
- `if (condizione) { ... }`: se la condizione è **vera** esegue il blocco, se è **falsa** lo salta. **Niente punto e virgola** dopo l'if.
- `digitalRead(pin)` **legge** un pin impostato come `INPUT`: `HIGH` se è collegato ai 5 V, `LOW` se è collegato al GND.
- Il codice si scrive **indentato**, con il tasto **Tab**.
- **Verifica** di firmware: martedì **17 novembre**, su tutte le dispense.